

VCS® MX QuickStart Guide

March 2017



Copyright Notice and Proprietary Information

© 2017 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Third-Party Software Notices

VCS®/VCSi™ and VCS® MX/VCS® MXi™ includes or is bundled with software licensed to Synopsys under free or open-source licenses. For additional information regarding Synopsys's use of free and open-source software, refer to the `third_party_notices.txt` file included within the `<install_path>/doc` directory of the installed VCS/VCS MX software.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <http://www.synopsys.com/Company/Pages/Trademarks.aspx>.

All other product or company names may be trademarks of their respective owners.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
690 E. Middlefield Road
Mountain View, CA 94043
www.synopsys.com

Contents

1. Overview	
About This Guide	1-2
Example Used in This Guide	1-2
Accessing VCS MX Documentation	1-3
Options for Help	1-3
2. Setting Up the Environment	
3. Setting Up the Simulation for DUT	
Analyzing the Design	3-3
Elaborating or Compiling the Design	3-5
Simulating the Design	3-5
Executing the Example	3-6
4. Setting Up the Simulation for Debug	
Analyzing and Elaborating the Design for Debug	4-4

Interactive Simulation Debug Using Verdi	4-5
Interactive Simulation Debug Using UCLI	4-9
Post-Processing Debug	4-10
Dumping FSDB	4-10
Post-Processing Debug Using Verdi	4-11
Viewing Schematics and Waveforms	4-12
Executing the Example	4-14
References	4-15
 5. Setting Up the Simulation for Monitoring Coverage	
Viewing Reports	5-5
Exclusions	5-6
Executing the Example	5-8
 Appendix A. Setting Up the Design for Simulation Profiling	
Running the profrpt Profile Report Generator	A-4
Executing the Example	A-5
 Appendix B. Setting Up the Simulation to Find X-Related Defects	
Viewing X-Propagation Log Files	B-4
Executing the Example	B-6
 Appendix C. Makefile	
 Appendix D. Additional References	

1

Overview

VCS®/VCS MX® functional verification solution is the primary verification solution used by most of the world's top 20 semiconductor companies. VCS MX provides the industry's highest performance simulation and constraint solver engines. In addition, the comprehensive VCS MX solution offers broad SystemVerilog support, verification planning, coverage analysis and closure, and native integration with Verdi® Automated Debug Platform, the industry's de-facto debug standard. VCS MX is uniquely positioned to meet designers' and verification engineers' needs to address the challenges and complexity of today's SoCs.

About This Guide

The guide describes the usage of basic VCS MX capabilities to simulate your design, perform basic debugging tasks, monitor coverage to measure the status of the verification process and its completeness, and check the design correctness using assertions.

This guide is intended for verification of both Verilog/SystemVerilog and VHDL designs and describes only the three-step flow. You can use two-step flow for compiling and simulating a design, if your design consists of only Verilog/SystemVerilog code. However, if you have also used VHDL in your design, use the three-step flow. For more information about the two-step and three-step flow, see the *VCS/VCS MX User Guide*.

It is assumed that the readers are familiar with HDL languages, such as Verilog, SystemVerilog, VHDL, and so on.

Example Used in This Guide

The design example used in this guide illustrates the various capabilities of VCS MX and consists of design files and scripts. This guide provides you the information about scripts included in this example, which you can run to set up your verification environment. For more information about the directory structure of the example, see the `README.txt` file provided with the example.

To download the example:

1. Log in to the SolvNet online support site using your SolvNet account at: <https://solvnet.synopsys.com/>

2. Click the **Documentation** tab and click **VCS MX**.
3. Click **Example Used in the QuickStart Guide**. This allows you to download the `MinSoc.tar.gz` file. Save the file to our working directory and untar the file.

Accessing VCS MX Documentation

VCS MX documentation is available in the SolvNet online support site. To access the product documentation, perform the following steps:

1. Log in to the SolvNet online support site using your SolvNet account at: <https://solvnet.synopsys.com/>
2. Click the **Documentation** tab and click **VCS MX**.

This page provides you access to PDF and HTML version of the VCS MX documentation.

Options for Help

To see the information about various VCS MX/Verdi options, run the following commands after installing VCS MX/Verdi:

- `% vcs -h | -help` - Lists descriptions of the most commonly used VCS MX compile time and runtime options.
- `% vcs -ID` - Returns information, such as VCS MX version and build date, VCS MX compiler version (same as VCS MX), and your work station name, platform, and host ID (used in licensing).

Note that when using the 64-bit installation of VCS MX, include the `-full64` option.

- `% verdi -h | -help` - Lists descriptions of the most commonly used Verdi options.
- `% verdi -id` - Returns information, such as Verdi version and build date, your work station name, platform, and host ID.

2

Setting Up the Environment

Before running Synopsys® tools, the Synopsys Common Licensing (SCL) software must be installed and configured, the license key file retrieved, and the license file environment variable defined.

You must define the LM_LICENSE_FILE license environment variable to point to your license file using the following command:

```
% setenv LM_LICENSE_FILE <path_to_the_license_file>
```

For detailed information about SCL installation and setup, see the [Synopsys Common Licensing Administration Guide](#) on [SolvNet®](#) online support site.

For the list of supported platforms for VCS MX and Verdi, see *VCS MX Release Notes* and *Verdi Release Notes* respectively.

Perform the following steps in C (CSH) shell to set up your environment:

- **VCS MX**

- Set the VCS_HOME environment variable to the path where VCS MX is installed:

```
% setenv VCS_HOME <VCS_installation_path>
```

- Set your UNIX PATH variable to \$VCS_HOME/bin:

```
% set path = ($VCS_HOME/bin $path)
```

- **Verdi**

- Set the VERDI_HOME environment variable to point to the installation directory:

```
% setenv VERDI_HOME <Verdi_installation_path>
```

- Set your UNIX PATH variable to \$VERDI_HOME/bin:

```
% set path = ($VERDI_HOME/bin $path)
```

Note:

Make sure to install the same release version of both VCS MX and Verdi.

3

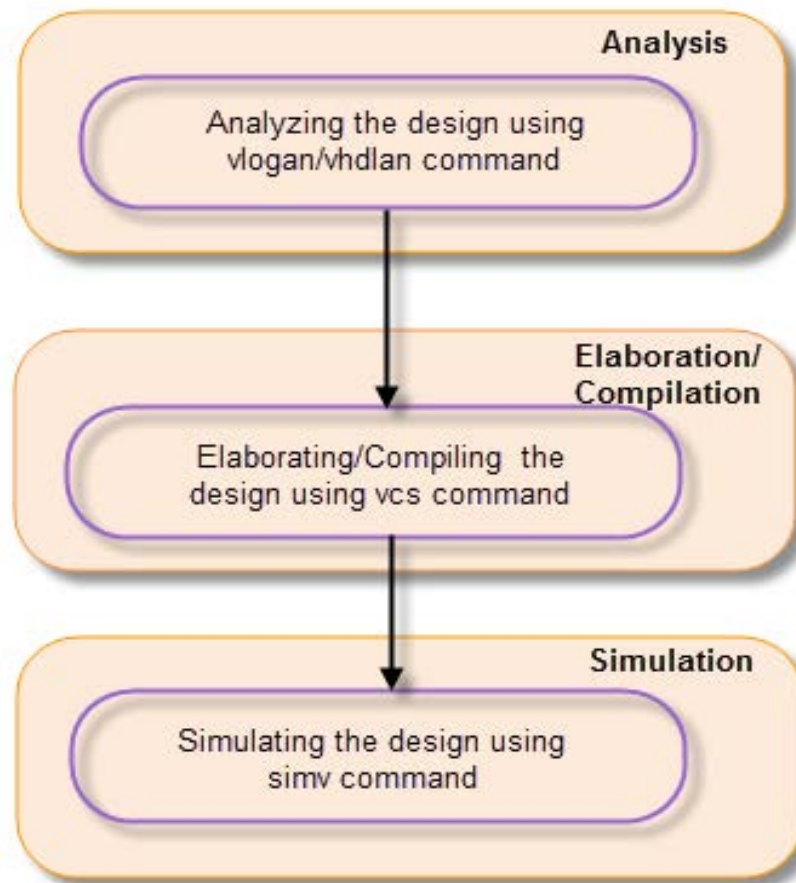
Setting Up the Simulation for DUT

Simulation is the predominant technique in verification. A verification engineer is required to compile a design and run simulations. The result would be either success or failure. If a simulation encounters a failure, the engineer will have to debug the failure, fix the design, test and rerun the simulations until all the tests are successful.

Verification execution encompasses all tasks from design compilation and simulation to debugging failures, and monitoring coverage of a design.

Simulating a design in VCS MX involves analyzing, elaborating and simulating the design code using the three-step flow, as illustrated in [Figure 3-1](#).

Figure 3-1 Basic Simulation Using Three-Step Flow



Note:

Apart from the three-step flow described in this document, if your design consists of only Verilog/SystemVerilog code, you can use the two-step flow (vcs and simv commands) for compiling and simulating the design. For more information about the two-step flow, see the *VCS User Guide*.

The commands that you can use for simulating a design are described in the following topics:

- [“Analyzing the Design”](#)
- [“Elaborating or Compiling the Design”](#)

- “Simulating the Design”
- “Executing the Example”

Analyzing the Design

During the **analysis** phase, you can analyze your VHDL, Verilog, and SystemVerilog files using the `vlogan` and `vhdlan` utility.

The `vlogan` utility analyzes the Verilog or the SystemVerilog portions of a design.

The syntax to use the `vlogan` command is as follows:

```
% vlogan -sverilog -work <library_name> -kdb
[vlogan_options] -f <source_list.f> -l <filename.log>
```

Where,

- `-sverilog` - Enables the analysis of SystemVerilog source code.
- `-work` - Option to specify the logical library name in which the analyzed code is added. Note that the mapping of logical library with the physical path of the library is present in the `synopsys_sim.setup` file. A sample `synopsys_sim.setup` file is as follows:

```
--Mapping default work directory
WORK > DEFAULT
DEFAULT : ./work
--Library Mapping
STATS_PKG : ./stat_work
MEM_PKG : ./mem_work
--Simulation variables
```

```
TIMEBASE = ps
TIME_RESOLUTION = 1NS
```

For more information about library mapping and the `synopsys_sim.setup` file, see section “Creating a `synopsys_sim.setup` File” in the *VCS MX User Guide*.

- `-f <source_list.f>` - To specify a file that contains a list of source files.
- `-kdb` - Enables you to create the KDB database, which is required to debug the design later using Verdi. Note that it is optional to use `-kdb` during regular analysis, however, it is mandatory to use `-kdb` when you require the debugging capabilities.
- `-l <filename.log>` - Captures the execution messages.

The syntax to use the `vhdlan` utility is as follows:

```
% vhdlan -work <library_name> -kdb [vhdlan_options] -f
<source_list.f> -l <filename.log>
```

Where,

- `-work`, `-kdb`, `-f` and, `-l` - Are described previously.

You can use the other analysis options in this command line to enable various other analysis capabilities. For all the options supported during analysis, see Appendix “Analysis Utilities” in the *VCS MX User Guide*.

Elaborating or Compiling the Design

During the **elaboration** phase, VCS MX uses the intermediate files generated during the analysis phase to elaborate (compile) and generate the binary executable called, `simv`. The `vcs` command enables you to elaborate the design and create a simulation executable.

The syntax to use the `vcs` command is as follows:

```
% vcs -sverilog -kdb -lca [elaboration_options]  
[design_unit] -l <filename.log>
```

Where,

- `-sverilog` - Enables SystemVerilog constructs specified in the IEEE Standard of SystemVerilog, IEEE Std 1800-2009.
- `design_unit` - Can be a module, entity, architecture, package body, or a configuration.
- `-l <filename.log>` - Captures the execution messages.

For information about the various elaboration options, see Appendix, “Elaboration Options” in the *VCS MX User Guide*.

Simulating the Design

During the **simulation** phase, you can simulate your design by executing the `simv` binary executable. This executable is generated during the elaboration phase. Use the following command line to simulate your design:

```
% simv [simulation/runtime_options] -l <filename.log>
```

For information about the various simulation or runtime options, see Appendix, “Simulation Options” in the *VCS MX User Guide*.

Executing the Example

After installing VCS MX and Verdi, download and copy the `Minsoc.tar.gz` file to your local work area.

Note:

Install only the 32 bit version of the VCS MX/Verdi as the pre-compiled `.so` file in MinSoc is compiled with 32 bit and not 64 bit.

To run the design example, execute the following steps:

1. Untar the example in your local work area:

```
% gtar -zxvf MinSoc.tar.gz
```

2. Change the directory to MinSoc:

```
% cd MinSoc
```

3. Source the `minsoc.setup` file.

```
% source minsoc.setup
```

4. Change to the `sim` directory.

```
% cd sim
```

5. To run the desired functionality, such as basic simulation, simulation for debug or simulation for monitoring coverage and so on, invoke the `make` command with the required functionality:

```
% make <run_required_functionality_of_target>
```


For example, to perform basic simulation of the design, use the following make command:

```
% make SOC_Verification
```

For the commands used in the make file, see [“Makefile”](#) .

4

Setting Up the Simulation for Debug

In a complex design, it is difficult to find the bugs associated with the logic at the HDL or testbench level. The process of debugging includes locating the design logic that is associated with an error, isolating the cause, and understanding how the design is supposed to behave and why it is not behaving that way. Design and verification engineers need sophisticated tools to find errors in the data produced by the simulator.

Verdi debug tool enables comprehensive debug for all design and verification flows. It includes powerful technology that helps you to comprehend complex and unfamiliar design behavior, and automate difficult and tedious debug processes.

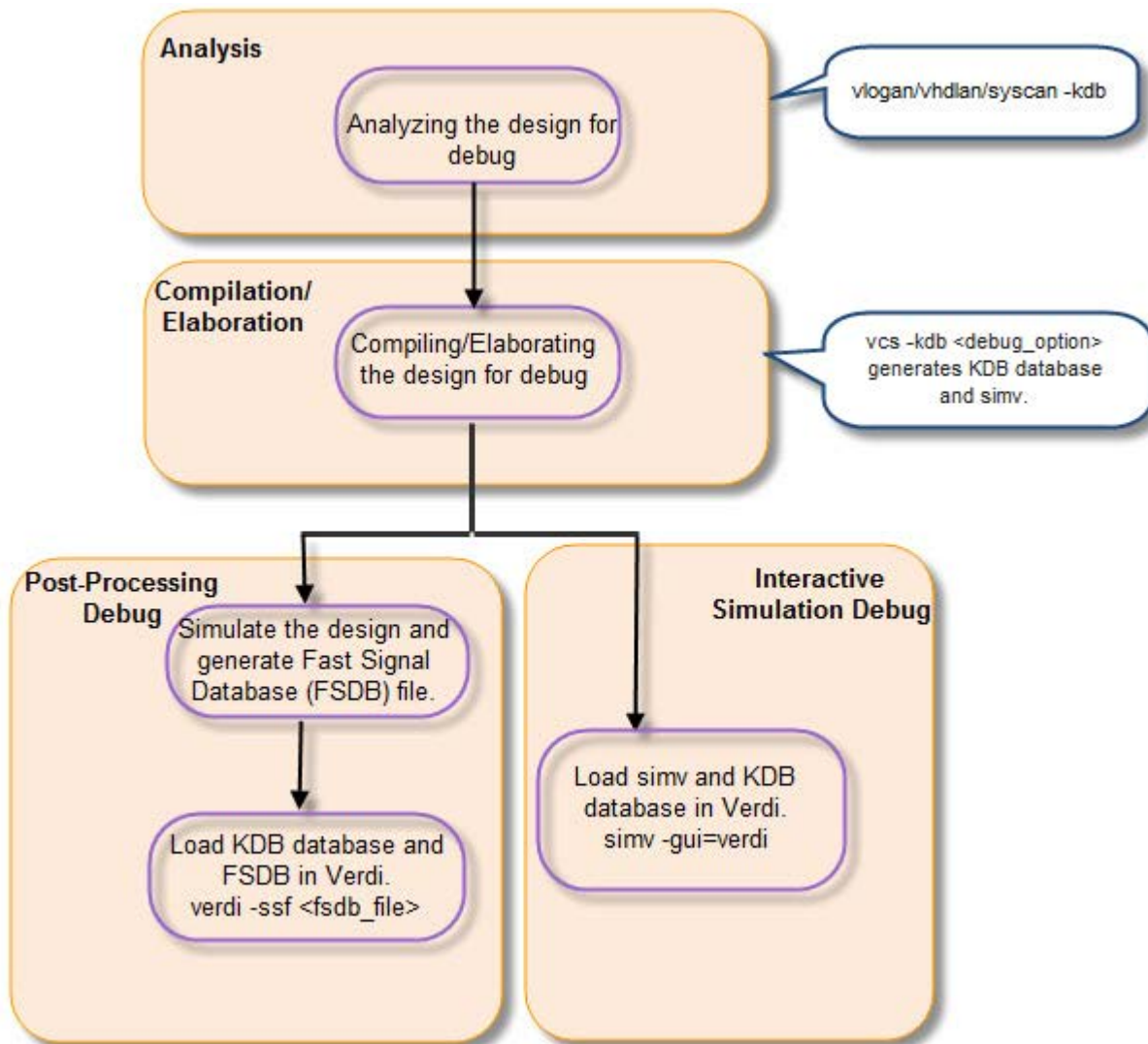
In addition to the standard features of a source code browser, schematics, waveforms, state machine diagrams, and waveform comparison, the Verdi tool includes advanced features for automatic tracing of signal activity using temporal flow views, assertion-based debug, and debug and analysis of transaction and message data.

You can debug your simulation using Verdi in the following modes:

- Interactive Simulation Debug Mode: Allows you to debug your design interactively.
- Post-Processing Debug Mode: Allows you to debug your design using a database.

The following flowchart shows the Verdi debug flow:

Figure 4-1 Verdi Debug Flow



This chapter includes the following topics:

- “Analyzing and Elaborating the Design for Debug”
- “Interactive Simulation Debug Using Verdi”
- “Post-Processing Debug”
- “Viewing Schematics and Waveforms”

- [“Executing the Example”](#)

Analyzing and Elaborating the Design for Debug

Analyzing the Design

For details on analyzing the design, see [“Analyzing the Design”](#).

Elaborating or Compiling the Design

To elaborate or compile the design for debug, use the following options with the vcs command:

- For post-processing debug, use the `-debug_access` compile-time option.
- For interactive debug, use the `-debug_access+all` compile-time option.

For details on the `-debug_access` option, see the *Optimizing Simulation Performance for Desired Debug Visibility with the -debug_access Option* section in the *VCS MX User Guide*.

For details on elaborating or compiling the design, see [“Elaborating or Compiling the Design”](#) section.

For all the options supported during elaboration, see *Elaboration Options* chapter in the *VCS MX User Guide*.

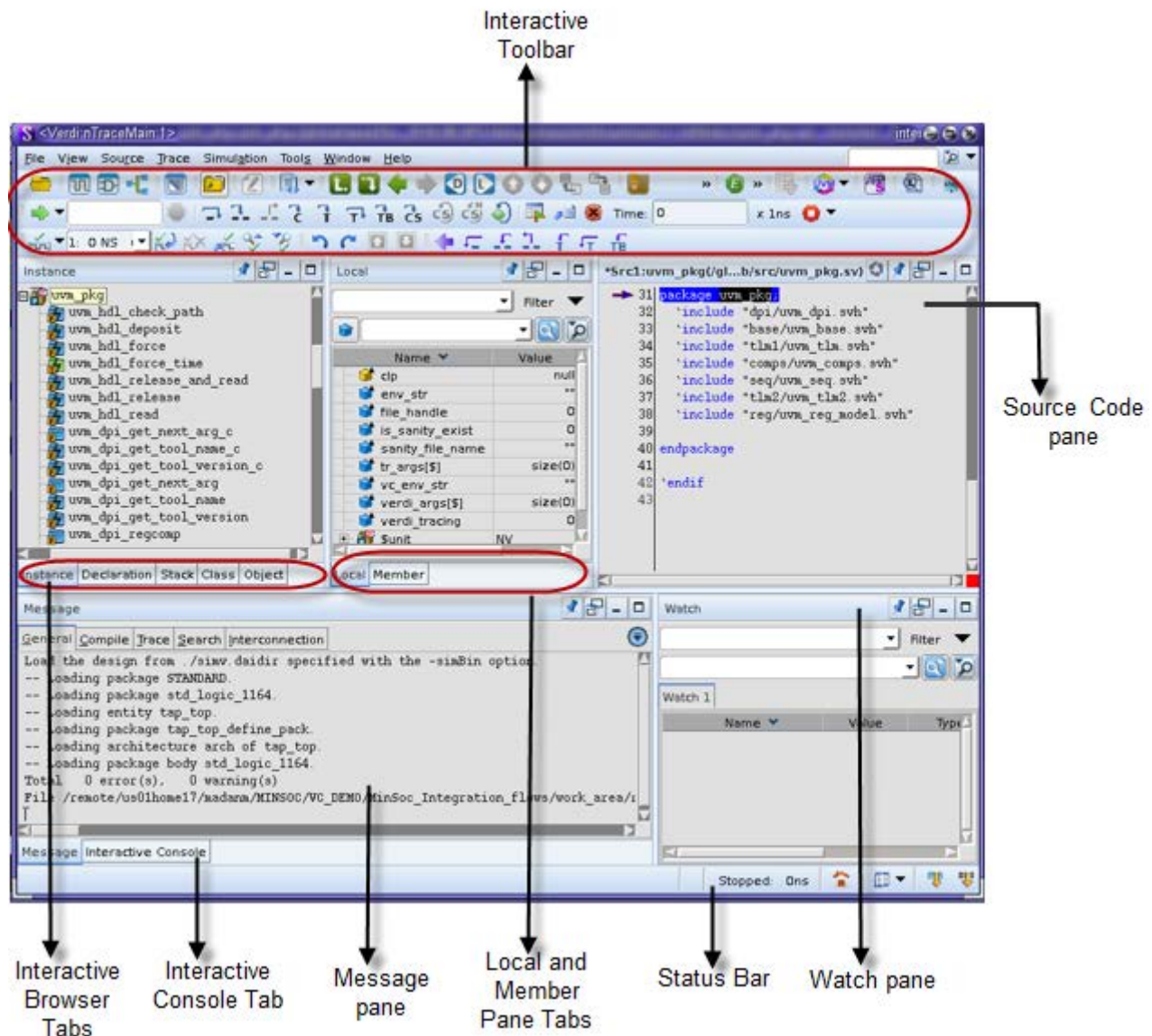
Interactive Simulation Debug Using Verdi

Verdi can be automatically invoked through the simulator command line option to perform interactive simulation debugging. Use the `-gui=verdi` option to invoke Verdi and perform interactive simulation debug.

```
% ./simv -gui=verdi [simulation_options]
```

On executing this command, Verdi interface opens, as shown in the following figure:

Figure 4-2 Verdi Interface



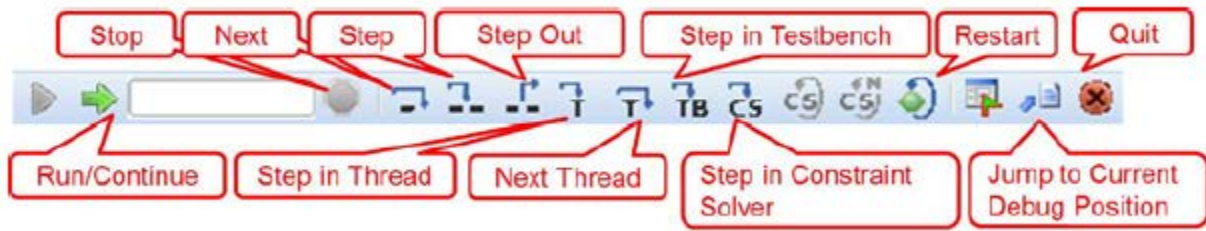
You can view the following panes in the interactive simulation debug mode:

- Instance Tab - Click the *Instance* tab to view the *Instance pane*. This pane displays the static instance tree. The names in the instance tree are in the “instance name (definition name)” format. Top modules (or scopes) are at the top-level of the tree.
- Stack Tab - Click the *Stack* tab to view the *Stack pane*. This pane displays the testbench dynamic hierarchy tree along with all the testbench threads and their status.
- Class Tab - Click the *Class* tab to view the *Class pane*. This pane displays all the classes defined in the design in a hierarchical view.
- Object Tab - Click the *Object* tab to view the *Object pane*. This pane displays current dynamic objects and its values.
- Local Tab - Click the *Local* tab to view the *Local pane*. This pane displays variables in a selected scope in the stack pane. This view is tied to the *Stack pane*, and the default view shows the variables of the current active thread.
- Member Tab - Click the *Member* tab to view the *Member pane*. This pane displays the members and values of a class object selected in the *Object pane*.
- Interactive Console Frame - The *Interactive Console* frame provides the interface to input the Unified Command Line Interface (UCLI) commands and shows the results of the simulator execution.
- Watch Pane - The *Watch pane* allows you to monitor the status of the variables during simulation.
- Status Bar - The *Status bar* shows the simulator status.

Simulation Control Commands

The frequently used simulation control commands are listed in the Interactive toolbar, as shown in the following figure, to direct the simulator how to execute the program.

Figure 4-3 Simulation Control Commands



You can advance the simulation to the statement where a function is called using the **Step** command (click the  icon or select the **Simulation > Step/Next > Step** command). The *Local tab* and *Watch tab* are refreshed accordingly.

When the **Next**  or **Step**  simulation control commands are used, and the scope of the current simulation belongs to a testbench or defined as a class object, all variables of the current simulation scope are shown automatically in the *Local tab*.

Use the *Watch tab* to view the current values of variables in the current simulation scope. Both the values and current scopes are updated as simulation time changes.

You can annotate the values to the variables on the *Source Code* frame using the **Source > Active Annotation** command. The values of the variables are displayed in the *Source Code* frame.

When you run a simulation interactively, the line where the simulation stopped is marked by the blue arrow in the *Source Code* frame.

You can set breakpoints to stop the simulation. Set the breakpoints by single-clicking or double-clicking the line number in the *Source Code* frame. Breakpoints execute each time a specified line is reached during simulation. A green flag  indicates that a breakpoint is set. Use **Simulation > Manage Breakpoints** command to open the Manage Breakpoints dialog box. This dialog box allows you to edit the breakpoints with more options.

The **Run Simulation** icon  or **Simulation > Run/Continue** command starts the simulator. If the simulator is invoked and stopped at a preset breakpoint (a flag and arrow combination indicates the current breakpoint and simulator position), invoke this command again to continue the simulation run.

For more information, see *Verdi SVTB Interactive Simulation Debug User Guide* and *Verdi and Siloti Command Reference Manual*.

Interactive Simulation Debug Using UCLI

Unified Command-line Interface (UCLI) provides a common set of commands for interactive simulation. UCLI is the default command-line interface for batch mode debugging in VCS MX.

The following command starts the simulation from the UCLI prompt:

```
% simv [simv_options] -ucli
```

At the `uc li` prompt, you can execute any UCLI command to debug or run the simulation.

For more information on using UCLI, see *Unified Command Line Interface User Guide* and *VCS MX User Guide*.

Post-Processing Debug

To perform post-processing debug, dump the Fast Signal Database (FSDB). The FSDB stores the simulation results, including transaction data and logged messages from SystemVerilog Testbench (SVTB) or other applicable languages, in an efficient and compact format that allows data to be accessed quickly.

Dumping FSDB

You can use either of the following methods to dump FSDB. To enable FSDB dumping, ensure that you specify the `-debug_access` option during elaboration.

- Specify FSDB dump system tasks in your code. For example,

```
initial
    begin
        $fsdbDumpvars;
        #12500
        $finish;
    end
```

For more information on the FSDB dump system tasks, see *Linking Novas Files with Simulators and Enabling FSDB Dumping Guide*.

- Use the `-ucli -do` option during simulation to import Tcl script with the UCLI dump command. For example,

```
% simv -ucli -do dump.tcl
```

Following are the contents of the `dump.tcl` file:

```
dump -file test.fsdb -type FSDB
dump -add <list_of_nids> -fid FSDB0
```

Where, `<list_of_nids>` is a list of scope or signals.

The above UCLI dump commands allow you to dump signal value changes of the specified instance and depth.

For more information on the UCLI dump commands, see *VCSMX User Guide/ Unified Command Line Interface User Guide*.

Post-Processing Debug Using Verdi

Perform the following steps to invoke post-processing debug mode:

1. Dump FSDB. For information on dumping FSDB, see [“Dumping FSDB”](#) section.
2. Analyze and elaborate your design using the `-kdb` option. For information on analyzing and elaborating your design, see [“Analyzing and Elaborating the Design for Debug”](#) section.
3. To generate FSDB, simulate your design.

```
%. /simv [simulation_options]
```

4. Load the design and the FSDB dump file into the Verdi platform.

```
% verdi -nologo -ssf <file_name.fsdb> &
```

Where, -ssf specifies FSDB file.

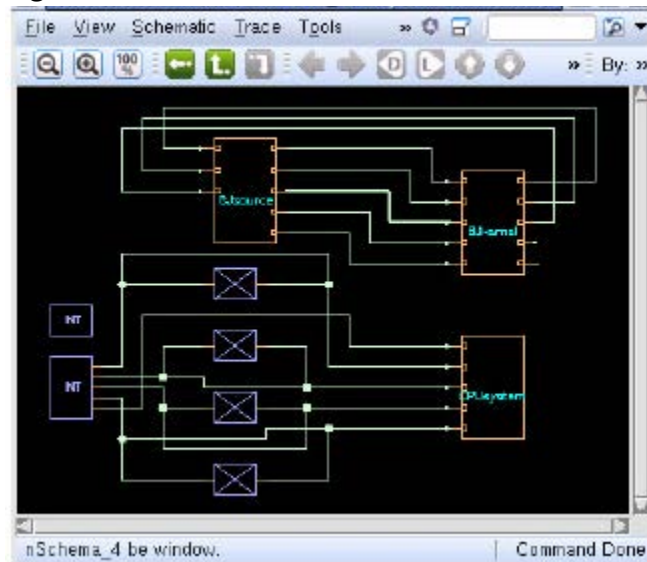
Verdi interface opens from which you can perform post-processing debug.

Viewing Schematics and Waveforms

nSchema is a schematic viewer and analyzer that generates interactive debug-specific logic diagrams showing the structure of selected portions of a design. The *nSchema* frame is used to display auto-generated schematics and logical diagrams.

Click the **New Schematic** icon  on the toolbar in the main window (or choose the **Tools -> New Schematic from Source -> New Schematic** command), to view the *nSchema* frame. The frame can be undocked to be a standalone *nSchema* window. A sample *nSchema* window is as follows:

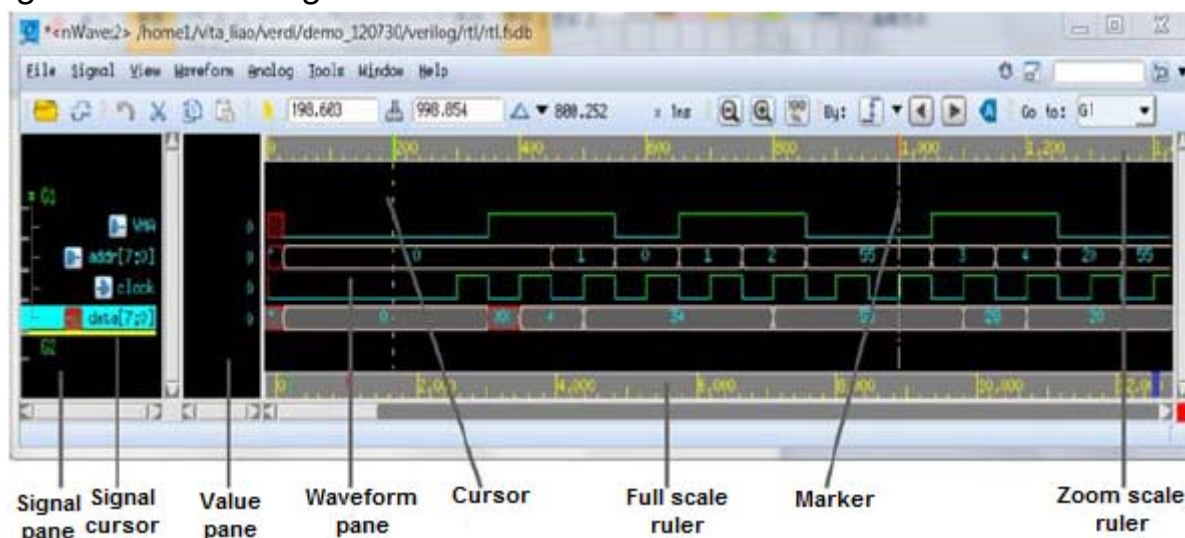
Figure 4-4 Viewing Schematic



nWave is a state-of-the-art graphical waveform viewer and analyzer that is fully integrated with the source code, schematic, and flow views of the Verdi platform.

You can open a new *nWave* frame from the main window by clicking the **New Waveform** icon  or choosing the **Tools -> New Waveform** command. An *nWave* frame can be released from the main window to become a stand-alone window by clicking the **Undock** toolbar icon. A sample *nWave* stand-alone window is as follows:

Figure 4-5 Viewing Waveforms



For more information on *nSchema* and *nWave*, see *Verdi User Guide and Tutorial Manual*.

Executing the Example

After setting up the design example (see the [“Executing the Example”](#) section), navigate to the `sim` directory.

To elaborate and simulate your design for interactive debug, execute the `SOC_Interactive_Debug` script from the `sim` directory as follows:

```
% make SOC_Interactive_Debug
```

To elaborate and simulate your design for post-processing debug, execute the `SOC_PP_Debug` script from the `sim` directory as follows:

```
% make SOC_PP_Debug
```

For the commands used in the make file, see [Appendix C](#), "Makefile".

References

Following are the SolvNet references for Interactive Debug:

- <https://solvnet.synopsys.com/retrieve/2504385.html>
- <https://solvnet.synopsys.com/retrieve/2325579.html>
- <https://solvnet.synopsys.com/retrieve/2514161.html>
- <https://solvnet.synopsys.com/retrieve/2386178.html>
- <https://solvnet.synopsys.com/retrieve/2499762.html>

5

Setting Up the Simulation for Monitoring Coverage

VCS MX contains features required to support coverage needs for today's designs. It enables you to generate coverage for a design, save it to a database, and generate and view coverage reports. Using VCS MX, you can generate the following types of coverage databases:

- Code Coverage

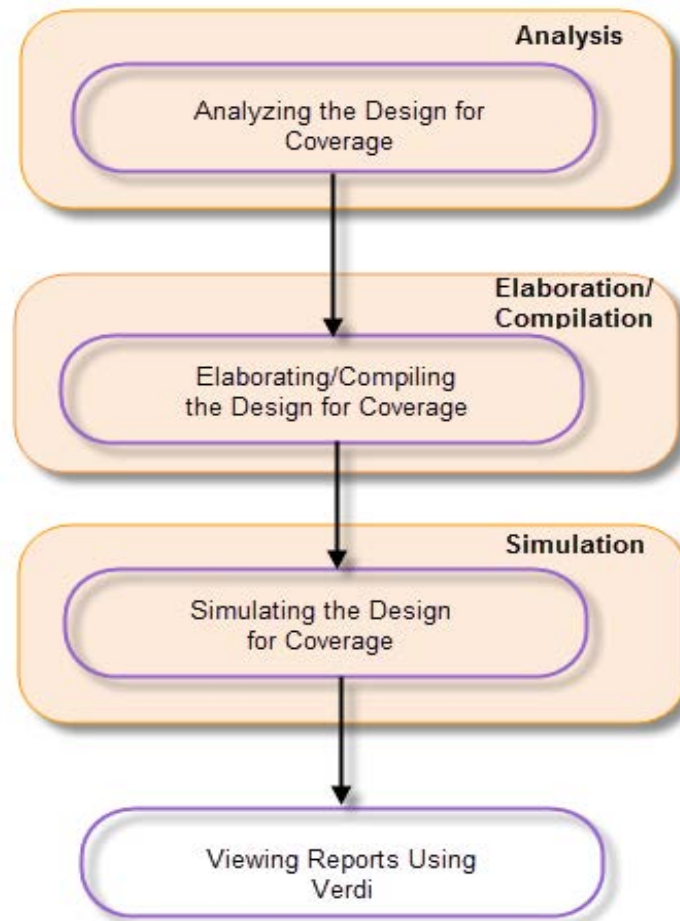
Code coverage monitors control-flow coverage and value coverage. Control-flow coverage includes line coverage and branch coverage. Value coverage includes condition coverage, toggle coverage, and FSM coverage.

- Functional Coverage

Functional coverage consists of covergroups and cover properties/assertions. Functional coverage is enabled by default as it only monitors your design, however, you can turn it off without having any effect on simulation.

To generate coverage for your design, use the three-step flow as illustrated in [Figure 5-1](#).

Figure 5-1 Setting Up the Simulation for Coverage



1. Analyzing the Design

It is not required to analyze your design again for coverage as there are no coverage-specific options for `vhdlan` and `vlogan`.

For details on this step, see Section [“Analyzing the Design”](#).

2. Elaborating or Compiling the Design

To elaborate or compile the design for coverage, use the `-cm` option with the `vcs` command. To specify the type of coverage for which you want VCS MX to compile your design, use the `-cm` option with any of the following arguments:

- `line`: Monitors the execution of procedural statements in the design.
- `cond`: Extracts and monitors combinations of values in conditional expressions.
- `tgl`: Monitors signal bit value toggles during simulation.
- `fsm`: Automatically extracts and monitors Finite State Machines in the design.
- `branch`: Monitors the execution of conditional control flow constructs.
- `assert`: Enables coverage monitoring for SystemVerilog Assertions.

If a testbench contains covergroups, VCS MX generates group coverage by default. For example, the following command generates group coverage automatically as if the testbench contains covergroups:

```
% vcs [design_unit] -cm line+cond+tgl+fsm+assert
```

You can control the scope of the coverage compilation using either pragmas or a `-cm_hier` configuration file. For details about pragmas and `-cm_hier`, see the *Coverage Pragmas* section and the *Compile Options for Coverage Metrics* section of the *Coverage Technology Reference Manual*.

For details on the *Elaborating or Compiling the Design* step, see Section [“Elaborating or Compiling the Design”](#).

3. Simulating the Design

To simulate the design for coverage, use the `-cm` option with the `simv` command. The `-cm` option specifies the type of coverage to collect during simulation. For example:

```
./simv -cm line+tgl+fsm+cond+assert
```

Only coverage types that are selected at compile time may be specified here, but you can specify a subset of those compile-time options.

For details on the *Simulating the Design* step, see Section [“Simulating the Design”](#).

After generating coverage, you can view reports using Verdi. For more details on this, see the [“Viewing Reports”](#) section.

Any complex design may have code or combinations that can never happen or that designers do not care about, then these can be excluded from coverage scores. Thus, you can focus on the coverage results from the design that can happen. For more details on this, see the [“Exclusions”](#) section.

To generate coverage and view reports, use the design example. For more details, see the [“Executing the Example”](#) section.

Viewing Reports

After the coverage database is generated, you can view a coverage report using Verdi. To view the report, invoke Verdi in the coverage mode using the following command:

```
% verdi -cov -covdir <coverage_database.vdb>
```

Where,

- `-cov` starts Verdi in the coverage mode.
- `-covdir` opens the database in the current directory.
- `<coverage_database.vdb>` is the coverage database. By default, it is `simv.vdb`.

The command displays the coverage report for monitoring, as illustrated in [Figure 5-2](#):

Figure 5-2 Verdi Coverage Report



For information about navigating through coverage data using Verdi, see Chapter “Using Verdi Coverage” in the *Verdi Coverage User Guide and Tutorial*.

Exclusions

To increase coverage by taking out exclusions that are not needed for a design, Verdi coverage provides the Exclusion Manager feature. When the design has a lot of code or combinations that never occur or there are parts of the design that a designer does not care about, then the exclusion features are used to exclude the above coverage targets from the coverage score. Thus, you can focus on the coverage results from the design that can happen.

The **Exclusion Manager** frame of Verdi Coverage combines all the exclusions from the different Verdi Coverage frames together. The **Exclusion Manager** frame is opened by default. You can mark an instance for exclusion by clicking an item in the **Hierarchy** tab of the **Summary** frame (see [Figure 5-3](#)).

Figure 5-3 Exclusion Manager

The screenshot displays the Exclusion Manager interface with three main panels:

- Summary Panel (Left):** A table listing modules and their coverage scores. A red callout points to the 'test_jukebox.sld coin1' instance.
- Code Editor (Center):** Shows Verilog code for a module. A blue callout points to the 'trko, daki' signals on line 5.
- CovDetail Panel (Right):** A table showing variable coverage details. A blue callout points to the 'trko[7:0]' variable.
- Exclusion Manager Panel (Bottom):** A table listing excluded items. Red and blue callouts point to the 'test_jukebox.sld coin1' instance and the 'trko[7:0]' variable, respectively.

Name	Score	Line	Tr
test_jukebox	43.62%	69.89%	
cd1	91.23%	100.00%	
clk1	76.82%	86.24%	
dsk1	75.90%	96.00%	
sld	43.69%	72.85%	
coin1	36.39%	67.42%	
trko	75.35%	89.66%	
trko_n	59.83%		
trko_n[0]	96.78%		
trko_n[1]	77.97%		
trko_n[2]	41.53%		

Line	Toggle	FSM	Condition	Branch	Assert
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					
14					

Variable	Type	Coverage
press	port	100
rst	port	100
state[2:0]	signal	66
trko[7:0]	port	66
trko_n[7:0]	signal	66
x_not	signal	0
y_not	signal	0

Name	Details	Annotation	Signature
test_jukebox.sld coin1	Instance: coin1		
test_jukebox.sld kp1			
Toggle: dsk_n			
Toggle: max_disk			
Toggle: trko[0]	1->0		
Toggle: trko[1]	0->1		
Toggle: trko[2]	0->1		
Toggle: trko[3]	1->0		
Toggle: trko_n			

For more details on Exclusion Manager, see Chapter, “Using Exclusion Manager” in the *Verdi Coverage User Guide and Tutorial*.

Executing the Example

After setting up the design example (see the [“Executing the Example”](#) section), navigate to the `sim` directory.

To elaborate and simulate your design for coverage, execute the `SOC_Coverage` script from the `sim` directory as follows:

```
% make SOC_Coverage
```

For the commands used in the make file, see [Appendix C, “Makefile”](#).

This script displays the Verdi Coverage report, as illustrated in [Figure 5-2](#).

A

Setting Up the Design for Simulation Profiling

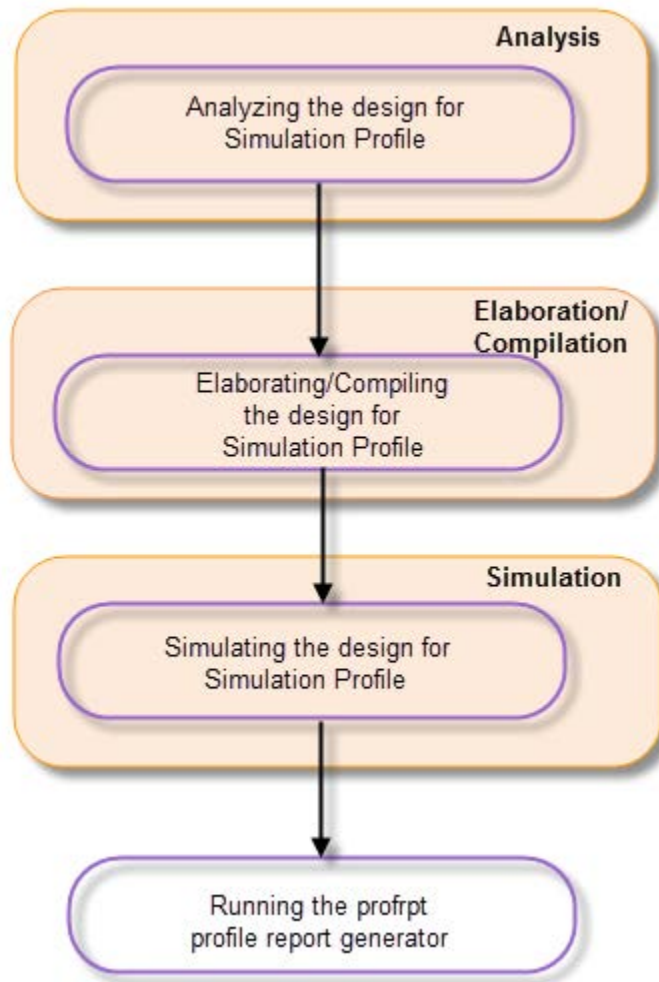
Large-scale designs regularly encounter issues, such as simulation performance degradation and increased turnaround time. The Unified Simulation Profiler provides a consistent method to analyze time and memory consumption of code written in Verilog, VHDL, SystemC, PLI, DPI, and DirectC. This helps in optimizing the simulation performance and reducing the turnaround time to improve verification efficiency.

The profiler provides both time and memory view. It can generate reports in HTML hyperlinked to the source code for high efficiency root cause analysis of memory leaks and coding missteps. The profiler can also generate text reports for integration into custom applications or reports. The reports are written by the `profrpt` profile report generator.

For details on Simulation Profiler, see the “The Unified Simulation Profiler” chapter in the *VCS MX User Guide*.

To set up the simulation for unified simulation profiler, you can use the three-step flow as illustrated in [Figure A-1](#):

Figure A-1 Setting Up the Simulation for Unified Simulation Profiler



1. Analyzing the Design

For details on this step, see Section [“Analyzing the Design”](#).

There is no need to analyze your design again for profiler as there are no profiler specific options for `vhdlan` and `vlogan`.

2. Elaborating or Compiling the Design

For details on the Elaborating or Compiling the Design step, see Section [“Elaborating or Compiling the Design”](#).

To compile your design for simulation profiler, use the `-simprofile` compile-time option. You can also use optional arguments with the `-simprofile` compile-time option.

For details on these arguments, see the “The Unified Simulation Profiler” chapter in the *VCS MX User Guide*.

3. Simulating the Design

For details on the Simulating the Design step, see Section [“Simulating the Design”](#).

You can use the `-simprofile` simulation option with a keyword argument or sub-option to specify the type of data VCS MX collects during simulation. These keyword arguments or sub-options are as follows:

- `time` the `time` argument specifies collecting CPU time profile information.
- `mem` the `mem` argument specifies collecting machine memory profile information.
- `noprof` tells VCS MX not to collect profiling information at simulation time. Synopsys recommends entering this simulation option and keyword argument or sub-option instead of simply omitting the `-simprofile` simulation option.
- `noreport` tells VCS MX to collect profile information at simulation time but not write the `profileReport.html` file or the `profileReport` directory after simulation.

After setting up the simulation for unified simulation profiler, you can run the profrpt profile report generator. For more details on this, see Section [“Running the profrpt Profile Report Generator”](#).

To set up the simulation for unified simulation profiler and run the profrpt profile report generator, see Section [“Executing the Example”](#).

Running the profrpt Profile Report Generator

You run the profrpt profile report generator with the profrpt command line. The syntax of this command line is as follows:

```
% profrpt simprofile_dir -view view1[+view2[+...]]  
[-h|-help][-format text|html|ALL][-output name]  
[-filter percentage][-snapshot [delta|incr|delta+incr]]  
[-timeline [dynamic_memory_type_or_class +...]]
```

Where,

- *simprofile_dir* - Specifies the profile database directory that VCS MX writes at simulation time. The default name is *simprofile_dir*.
- -view *view1*[+*view2*[+...]] - Specifies the views you want to see in the reports. The option is mandatory.

The profrpt output is the HTML or text files for all these views, including machine memory stack traces.

You must enter the -view option on the profrpt command line.

The views you can specify with the -view option depends on the type of report that profrpt is writing, which depends on the argument to the -simprofile simulation option.

Some of the arguments and the views that you can specify are `time_summary`, and `mem_summary`.

For more information on all the arguments and sub-options, see the “The Unified Simulation Profiler” chapter in the *VCS MX User Guide*.

Executing the Example

After setting up the design example (see the [“Executing the Example”](#) section), navigate to the `sim` directory.

To elaborate and simulate your design for simulation profile, execute the `SOC_Profile` script from the `sim` directory as follows:

```
% make SOC_Profile
```

For the commands used in the make file, see [Appendix C, “Makefile”](#).

You can run the `profrpt` profile report generator to generate profiler reports in text and HTML formats.

B

Setting Up the Simulation to Find X-Related Defects

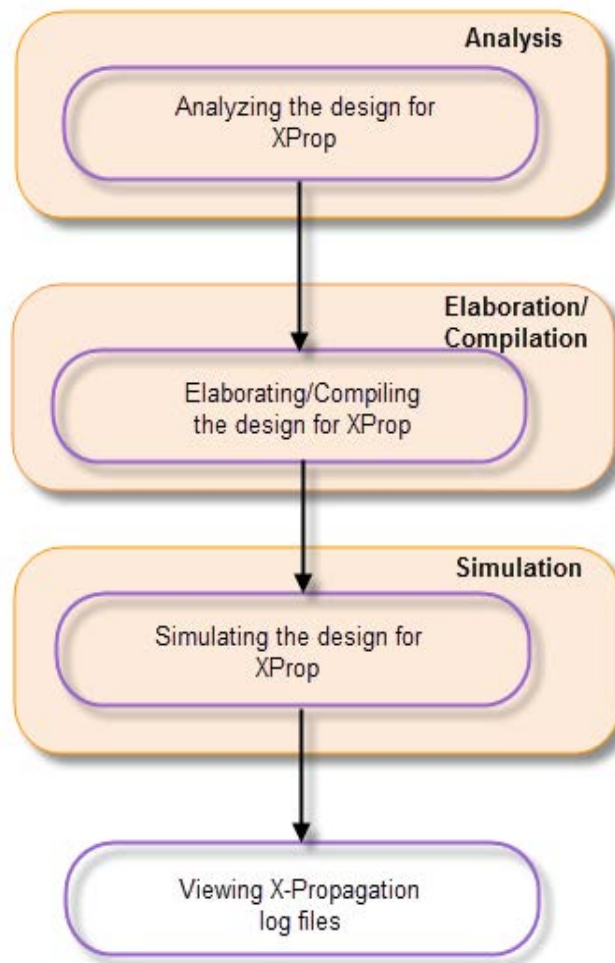
Verilog and VHDL RTL simulators ignore the uncertainty of X-valued control signals and assign predictable output values because of the semantic limitations. As a result, RTL simulations often fail to detect design problems related to the lack of X-Propagation (Xprop). However, these same design problems can be detected in gate-level simulations, and often many gate-level simulations must be run only to debug X-related issues.

VCS MX simulator provides an add-on Xprop simulation semantic mode that allows standard RTL simulations to expose problems related to X being propagated. This saves engineers time and effort in debugging differences between RTL and gate-level simulation results.

For details on Xprop, see “Using X-Propagation” chapter in the *VCS MX User Guide*.

To set up the simulation for Xprop, you can use the three-step flow: as illustrated in [Figure B-1](#).

Figure B-1 Setting Up the Simulation for Xprop



1. Analyzing the Design

For details on this step, see Section [“Analyzing the Design”](#).

There is no need to analyze your design again for Xprop as there are no Xprop-specific options for `vhd lan` and `v logan`.

2. Elaborating or Compiling the Design

For details on the Elaborating or Compiling the Design step, see Section [“Elaborating or Compiling the Design”](#) .

To compile your design in the Xprop mode, use the `-xprop` option with the `vcs` command. The `-xprop` option without an argument enables Xprop in the entire design and specifies the merge mode at simulation time. By default, VCS MX uses the `tmerge` merge mode during simulation. However, you can specify a different merge mode at either compile time or simulation time. The keyword argument `flowctrl` is added for VHDL instrumentation.

You can use the following arguments for the `-xprop` option:

- `<xprop_config_file>`
- `tmerge`
- `xmerge`
- `flowctrl`

For details on these arguments, see “Using X-Propagation” chapter in the *VCS MX User Guide*.

3. Simulating the Design

For details on the Simulating the Design step, see Section [“Simulating the Design”](#) .

Use the following option to generate an Xprop instrumentation report that displays the instrumentation status of every module instance in a design:

```
-report=xprop[+exit]
```

Where,

- `xprop` Generates an Xprop instrumentation report named as `xprop_config.report` and the simulation continues.
- `xprop+exit` Generates an Xprop instrumentation report named as `xprop_config.report` and the simulation is terminated.

After simulating your design, you can view the log files. For more details on this, see Section [“Viewing X-Propagation Log Files”](#).

To detect uninitialized memory during RTL simulation using Xprop, see Section [“Executing the Example”](#).

Viewing X-Propagation Log Files

When you compile a design with Xprop enabled, VCS MX generates log files `xprop.log` and `xprop_vhdl.log` for Verilog and VHDL respectively. All the statements considered for Xprop instrumentation (whether or not the statements are instrumented), and the reasons for statements not being instrumented are recorded in these log files.

Following is the format of a report entry in `xprop.log`/
`xprop_vhdl.log` files:

```
filename:line_number YES|NO ["reason" (primary_line)]
```

Where,

- `filename` - name of the source file containing a statement being considered for Xprop instrumentation.
- `line_number` - line number that corresponds to the start of the statement.

- YES|NO - Xprop instrumentation status of the statement.
- reason - The reason for the statement not being instrumented. This is issued only when the Xprop instrumentation status is "NO".
- primary_line - The line number of the statement containing the actual construct not being instrumented. This is issued only when the Xprop instrumentation status is "NO".

xprop.log example:

```
decode.v:3 YES
```

```
decode.v:7 NO "prevented by sub-statement" (12)
```

```
decode.v:16 NO "delay statement" (17)
```

```
decode.v:18 YES
```

```
decode.v:20 NO "a dynamic object" (22)
```

xprop_vhdl.log example:

```
Test.vhd:22 YES
```

```
Test.vhd:35 YES
```

```
Test.vhd: 44 NO assignment has non-null delay (41)
```

The Xprop statistics are presented at the end of the xprop.log file. The statistics consist of the number of assignment statements considered for Xprop instrumentation, the number of statements instrumented, and the ratio of those two numbers (instrumented/instrumentable) which represents the percentage of the design instrumented for Xprop.

For example:

```
=====
X P R O P   S T A T I S T I C S
instrumentable assignments:      7
instrumented assignments:       5
instrumentation success rate:   71%
```

Executing the Example

After setting up the design example (see the [“Executing the Example”](#) section), navigate to the `sim` directory.

To elaborate and simulate your design for Xprop, execute the `SOC_Xprop_RTL` script from the `sim` directory as follows:

```
% make SOC_Xprop_RTL
```

For the commands used in the make file, see [Appendix C, “Makefile”](#).

The following error is reported:

```
UVM_ERROR /remote/us044/xyz/MinSoc_Example/design/tb/
uvm_tb/minsoc_scoreboard.sv(303) @ 4187560:
uvm_test_top.env.sb [uart_scoreboard] DCE_TX_DCE_RX_COMP :
Mismatch between dce_tx value is 56 and dce_rx value is 00
```

This error is reported when running the simulation with Xprop simulation semantics due to an uninitialized memory in the design/ethmac/src/eth_spram_256x32.v file. However, if the same setup is run using RTL semantics (without Xprop), this error would be masked due to the optimistic behavior of RTL.

As illustrated in this example, Xprop helps you to detect the uninitialized memory earlier in the cycle, that is, during RTL simulation instead of gate-level simulation.

This design defect is fixed with the +define+MEM_INITIALIZE_FIX define in the SOC_Xprop_RTL_fixed script, which is located in the sim directory. You can use the elaboration and simulation commands mentioned in this script to simulate your design successfully.

C

Makefile

The `Makefile` that is packaged as part of the design example includes the following VCS MX and Verdi command lines:

```
dirs := or1200 tap_top tc_top memory_model clock_manager ethmac adbg_top uart
top tb
```

```
SOC_Verification: clean ana comp sim
```

```
SOC_Interactive_Debug: clean ana comp_interactive_debug sim_debug_interactive
```

```
SOC_PP_Debug: clean ana_pp comp_pp_debug sim debug
```

```
SOC_Coverage: clean ana comp_cov sim_cov report_cov
```

```
SOC_Profile: clean ana comp_prof sim_prof report_prof
```

```
SOC_Xprop_RTL: clean ana comp_xprop sim
```

```
SOC_Xprop_RTL_fixed: clean ana_xprop comp_xprop sim
```

```
clean:
```

```
\rm -rf simv* csrc* ${ROOT}/libs *.log ucli.key vhdlanLog verdiLog
novas.fsdb novas.rc novas.conf profileReport* simprofile_dir vdCovLog
vc_hdrs.h .vlogansetup.args inter.fsdb mkdir
```

```

    $(foreach i, $(dirs), mkdir -p ${ROOT}/libs/${i})

ana_core:
    vlogan -l an_or1200.log -sverilog -kdb -work or1200 -f or1200.f
    vhdlan -l an_tap_top.log -kdb -work tap_top -f tap_top.f
    vlogan -l an_tc_top.log -sverilog -kdb -work tc_top -f minsoc_tc_top.f
    vlogan -l an_memory_model.log -sverilog -kdb -work memory_model -f
    minsoc_memory_model.f
    vlogan -l an_adbg.log -sverilog -kdb -work adbg_top -f adbg_top.f
    vlogan -l an_clock_manager.log -sverilog -kdb -work clock_manager -f
    minsoc_clock_manager.f
    vlogan -l an_ethmac.log -sverilog -kdb -work ethmac -f ethmac.f
    vlogan -l an_intf.log -sverilog -kdb -work intf -f intf.f
    vlogan -l an_uart.log -sverilog -kdb -work uart -f uart.f
    +define+UART_HOST_IF
    vlogan -l an_top.log -sverilog -kdb -work top -f minsoc_top.f
    vlogan -l an_uvm.log -sverilog -kdb -work tb -ntb_opts uvm
    vlogan -l an_tb.log -sverilog -kdb -work tb -ntb_opts uvm -f tb.f

ana: ana_core

ana_pp: ana_core
    vlogan -l an_tb.log -sverilog -kdb -work tb -ntb_opts uvm -f tb.f
    +define+DUMP

ana_xprop: ana_core
    vlogan -l an_ethmac.log -sverilog -kdb -work ethmac -f ethmac.f
    +define+MEM_INITIALIZE_FIX

comp:
    vcs minsoc_tb_top test -ntb_opts uvm -kdb -lca
    +warn=noLCA_FEATURES_ENABLED -l comp.log

comp_pp_debug:

    vcs minsoc_tb_top test -ntb_opts uvm -debug_access -kdb -lca
    +warn=noLCA_FEATURES_ENABLED -l comp.log

comp_interactive_debug:

    vcs minsoc_tb_top test -ntb_opts uvm -debug_access+all -kdb -lca
    +warn=noLCA_FEATURES_ENABLED -l comp.log

comp_cov:

    vcs minsoc_tb_top test -ntb_opts uvm -kdb -lca
    +warn=noLCA_FEATURES_ENABLED -l comp.log -cm
    line+cond+tgl+fsm+branch+assert

```

comp_prof:

```
vcs minsoc_tb_top test -ntb_opts uvm -kdb -lca  
+warn=noLCA_FEATURES_ENABLED -l comp.log -simprofile time
```

comp_xprop:

```
vcs minsoc_tb_top test -ntb_opts uvm -kdb -lca  
+warn=noLCA_FEATURES_ENABLED -l comp.log -xprop=minsoc_xprop.cfg
```

sim:

```
./simv +file_name=${ROOT}/sim_files/eth.hex +firmware_size=12268  
+UVM_VERBOSITY=UVM_HIGH +UVM_TESTNAME=eth_test -l simv_eth.log
```

sim_debug_interactive:

```
./simv -gui=verdi +file_name=${ROOT}/sim_files/eth.hex  
+firmware_size=12268 +UVM_VERBOSITY=UVM_HIGH +UVM_TESTNAME=eth_test -l  
simv_eth.log
```

sim_cov:

```
./simv +file_name=${ROOT}/sim_files/eth.hex +firmware_size=12268  
+UVM_VERBOSITY=UVM_HIGH +UVM_TESTNAME=eth_test -l simv_eth.log -cm  
line+cond+tgl+fsm+branch+assert +enable_coverage=1
```

sim_prof:

```
./simv +file_name=${ROOT}/sim_files/eth.hex +firmware_size=12268  
+UVM_VERBOSITY=UVM_HIGH +UVM_TESTNAME=eth_test -l simv_eth.log  
-simprofile noreport
```

debug:

```
verdi -nologo -ssf novas.fsdb &
```

report_cov:

```
verdi -cov -covdir simv.vdb
```

report_prof:

```
profrpt simprofile_dir -view time_summary+time_mod
```

D

Additional References

In addition to the VCS MX functionalities introduced in this guide, VCS also offers you the following functionalities that help improve verification process:

- **Verification Planner** - The Verification Plan is to the Verification engineer as the functional specification is to the RTL designer. Verification Planning is the process of developing documents that capture verification goals and how to reach those goals and provide a framework for tracking and reporting progress through the life of the verification effort.
There are two types of documents that make up a complete Verification Plan. First, a verification document - the Verification Approach Specification - in MS Word, FrameMaker, or some other word processing tool. The second part is an executable Verification Plan. The partitioning between the two is largely determined by the automation features available in the executable verification planning tool. The goal should be to utilization automation whenever feasible to reduce the overall verification

planning effort, improve progress tracking and metric quality. The Verification Approach Specification defines the overall approach to verifying the Design Under Test (DUT). It focuses on the what DUT features need to be defined, the approach to verification and the top-level metrics being used to validate completeness. The accompanying Executable Verification Plan breaks down these top-level metrics to the desired level of detail to support an automated back annotation flow. The term Verification Plan is used when referring to either one (or both) as aspects of the complete system. For more information, see the *Verification Planner User Guide*.

- **SystemC Cosimulation** - VCS MX SystemC cosimulation interface enables VCS MX and SystemC modeling environment to work together, when simulating a system described in Verilog, VHDL, and SystemC languages. For more information about the SystemC cosimulation interface, see the *VCS MX/VCS MXi SystemC User Guide*.
- **Mixed-Signal Verification** - The VCS MX and CustomSim cosimulation feature provides mixed-signal, mixed-HDL verification solution. This feature enables you to simulate a design, which is described in SPICE, Verilog-HDL (Verilog), and VHDL. For more information about mixed-signal verification, see chapter “VCS MX and CustomSim Cosimulation” in the *VCS MX User Guide*.
- **Partition Compile** - The VCS MX Partition Compile feature allows you to achieve significantly faster turnaround time by allowing you to recompile portions of the design that has changed and reuse the libraries for the unchanged modules already compiled earlier. For more information about partition compile, see chapter “Partition Compile” in the *VCS MX LCA User Guide*.
- **Precompiled IP** - In verifying complex designs, the testbench environment tends to become bulky, which means the simulator

tool requires more time to compile and elaborate samples on the design and test bench. Traditional tools like incremental compilers can no longer meet the complex verification requirements. For verification, the compilation time can be methodically controlled using the VCS MX Precompiled IP (PIP) feature.

Precompiled IP involves the following:

- Fully compiling the design or testbench rooted at the topmost module/configuration of the IP.
- Generating a shared object and debug database needed for simulation.

For more information about precompiled IP, see chapter “Precompiled IP” in the *VCS MX LCA User Guide*.

- **SystemVerilog Assertions** - Typically, the method of verifying a design involves writing tests and running simulations to exercise the tests. Monitors are spread throughout the tests, the design model, and the post-processing software or scripts. There is no concise description of the behavior that is being tested. This unorganized way creates several challenges.

Using SystemVerilog Assertions (SVA) you can specify how you expect a design to behave and enforce VCS MX to display messages when the design does not behave as specified. For more information about SystemVerilog, see chapter “Using SystemVerilog Assertions” in the *VCS MX User Guide*.

- **MVSIM Native Mode** - MVSIM Native Mode enables you to specify the UPF based power-intent of your design directly to VCS and generate a simulation model, which contains all power-objects directly instrumented in it. For more information, see *VCS Native Low Power (NLP) User Guide*.

- **Native Certitude** - Certitude is a functional qualification tool. This tool enhances simulation-based functional verification by providing measures and feedback to assist in improving the quality of the verification environment (VE).

To detect errors, the functional verification environment must ensure that each error is activated, propagated, and then detected.

With Native Certitude (VCS MX and Certitude integration), you can generate VCS MX and Certitude databases with the same compilation. By eliminating the dependency on different parsers, the fault-aware simulation executable can be generated without having to separately prepare the design for testbench qualification. For more information about Native Certitude, see chapter “Integrating VCS MX With Certitude” in the *VCS MX User Guide*.

Index

A

assert 5-3

B

branch 5-3

C

-cm 5-3

-cm_dir 5-3

-cm_hier 5-4

cond 5-3

-cov 5-5

-covdir 5-5

Coverage 5-1

coverage 5-2

coverage mode 5-5

covergroups 5-3

D

Design Example 5-8

E

Exclusion Manager 5-6

exclusions 5-6

F

fsm 5-3

FSM coverage 5-1

G

group coverage 5-3

H

Hierarchy tab 5-6

L

line 5-3

O

-o 5-3

P

pragmas 5-4

S

simv 3-5

Summary frame 5-6

T

tgl 5-3

three-step flow 5-2, A-2

toggle coverage 5-1

U

URG 5-4

V

vcs B-3

Verdi 5-4, 5-5

Verdi Coverage 5-6

vhdlan 3-3

vlogan 3-3